408 计算机组成原理：数据的表示和运算（考研复习笔记）

这一章是408计算机组成原理中的重要计算章节，特点是**公式多、规则多，但常见题型比较固定**。

尤其是原码、反码、补码、移位、溢出判断、定点数运算、浮点数运算等内容，掌握固定步骤后，比较容易在考试中拿到基础分。

这份笔记按照 **基本概念 → 核心公式 → 典型例题 → 解题步骤 → 易错点** 整理，重点放在能够直接用于解题的知识上。

本章考点总览

知识模块	主要考点	优先级
数制与编码	进制转换、BCD、ASCII	重点
定点数表示	原码、反码、补码、移码	必会
数据类型与转换	有符号、无符号、扩展、截断	必会
移位运算	算术移位、逻辑移位	必会
定点加减法	补码运算、溢出判断	必会

一、数制与进制转换

1. 常见进制

进制	基数	使用数字	示例
二进制	2	0、1	1011_2
八进制	8	0～7	13_8
十进制	10	0～9	11_{10}
十六进制	16	0～9、A～F	B_{16}

重要对应关系

$$\boxed{1\text{ 位八进制} = 3\text{ 位二进制}}$$

$$\boxed{1\text{ 位十六进制} = 4\text{ 位二进制}}$$

例如：

$$(10110110)_2$$

按4位分组：

$$1011 \quad 0110$$

对应：

$$\boxed{B6_H}$$

2. 二进制转十进制

整数部分按权展开：

$$(101101)_2$$

$$= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1$$

$$= 32 + 8 + 4 + 1$$

$$\boxed{45}$$

小数部分：

$$(0.101)_2$$

$$= 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}$$

$$= 0.5 + 0.125$$

$$\boxed{0.625}$$

3. 十进制转二进制

整数部分：**除2取余，倒序排列。**

小数部分：**乘2取整，顺序排列。**

例如：

$$0.625 \times 2 = 1.25$$

取1，剩余0.25。

$$0.25 \times 2 = 0.5$$

取0。

$$0.5 \times 2 = 1.0$$

取1。

因此：

$$\boxed{0.625_{10} = 0.101_2}$$

注意：部分十进制小数不能用有限位二进制精确表示，例如0.1。

二、原码、反码、补码、移码 必会

这是全章必须掌握的基础。

1. 原码

原码的规则：

- 最高位为符号位。
- 0表示正数。
- 1表示负数。
- 其余位表示数值的绝对值。

例如用8位表示：

$$+5 = 00000101$$

$$-5 = 10000101$$

n位原码表示范围

$$\boxed{-(2^{n-1}-1) \sim +(2^{n-1}-1)}$$

8位原码：

$$\boxed{-127 \sim 127}$$

原码存在两个零：

$$+0 = 00000000$$

$$-0 = 10000000$$

2. 反码

正数反码与原码相同。

负数反码：

符号位不变，数值位按位取反。

例如：

$$-5_{\text{原}} = 10000101$$

符号位1不变，其余取反：

$$\boxed{-5_{\text{反}} = 11111010}$$

反码也存在正零和负零。

3. 补码

正数补码与原码相同。

负数补码：

原码符号位不变、数值位取反，再加1。

更通用的理解是：

$$\boxed{[-x]_{\text{补}} = 2^n - x}$$

这里 $x > 0$，且结果按照n位二进制编码。

例如8位补码表示 -5：

$$2^8 - 5 = 251$$

$$251 = 11111011_2$$

所以：

$$\boxed{[-5]_{补} = 11111011}$$

8位补码范围

$$\boxed{-128 \sim 127}$$

一般n位补码范围：

$$\boxed{-2^{n-1} \sim 2^{n-1} - 1}$$

补码只有一个零：

$$00000000$$

最容易考的特殊值

8位补码：

$$10000000 = -128$$

而不是负零。

$$11111111 = -1$$

$$01111111 = 127$$

4. 补码快速求真值

例如8位补码：

$$11110110$$

最高位为1，是负数。

方法一：取反加1。

$$11110110$$

取反：

$$00001001$$

加1：

$$00001010 = 10$$

所以真值：

$$\boxed{-10}$$

方法二：直接减去 2^n。

$$11110110_2 = 246$$

$$246 - 256 = -10$$

因此：

$$\boxed{x = -10}$$

考试推荐第二种方法，尤其适合十六进制补码。

例如：

$$FC_H = 252$$

按8位补码解释：

$$252 - 256 = -4$$

5. 移码

移码通常用于表示浮点数的阶码。

常见定义：

$$\boxed{[x]_{移} = x + 2^{n-1}}$$

其中n是编码总位数，此时偏置值为 2^{n-1}。

例如用8位、偏置128的移码表示 -5：

$$-5 + 128 = 123$$

$$\boxed{01111011}$$

重要区别

IEEE 754浮点数阶码也采用偏置编码，但常用偏置不是 2^{n-1}，而是：

$$\boxed{Bias = 2^{k-1} - 1}$$

例如单精度8位阶码：

$$Bias = 127$$

不要把教材中的普通移码公式与IEEE 754阶码直接混为一谈。

三、四种机器数表示方法对比

以8位表示 $+5$ 和 -5：

编码	+5	-5
原码	00000101	10000101
反码	00000101	11111010
补码	00000101	11111011
移码（偏置128）	10000101	01111011

n位整数表示范围

编码	最小值	最大值	零的数量
原码	$-(2^{n-1}-1)$	$2^{n-1}-1$	2
反码	$-(2^{n-1}-1)$	$2^{n-1}-1$	2
补码	-2^{n-1}	$2^{n-1}-1$	1
移码（偏置 2^{n-1}）	-2^{n-1}	$2^{n-1}-1$	1

互动练习：8位补码转换

观察负数的机器数编码 -5（十进制）

原码	10000101
反码	11111010
补码	11111011
移码（偏置128）	01111011

范围限制为 -127～127，以便四种编码都能表示。这里使用固定8位整数格式。

四、有符号数与无符号数 必会

相同的二进制位模式，按照不同数据类型解释，数值可能完全不同。

例如：

$$11111111$$

如果按8位无符号整数解释：

$$\boxed{255}$$

如果按8位补码有符号整数解释：

$$\boxed{-1}$$

1. 表示范围

n位无符号整数：

$$\boxed{0 \sim 2^n - 1}$$

n位补码有符号整数：

$$\boxed{-2^{n-1} \sim 2^{n-1} - 1}$$

例如8位：

类型	范围
unsigned	0～255
signed（补码）	-128～127

2. 符号扩展

有符号补码从低位数扩展到高位数：

复制原来的最高符号位。

例如8位 -5：

$$11111011$$

扩展到16位：

$$\boxed{1111111111111011}$$

数值仍然是 -5。

3. 零扩展

无符号数扩展时，高位补0。

例如：

$$11111011$$

从8位零扩展到16位：

$$\boxed{0000000011111011}$$

表示251。

必背结论

- 有符号补码：符号扩展。
- 无符号数：零扩展。
- 截断高位可能造成数值改变。
- 相同位模式，类型不同，含义可能不同。

五、移位运算 必会

移位运算主要分为：

- 逻辑移位
- 算术移位
- 循环移位

1. 逻辑移位

通常用于无符号二进制数。

规则：

- 左移：低位补0。
- 右移：高位补0。

例如：

$$00110100$$

逻辑左移1位：

$$01101000$$

逻辑右移1位：

$$00011010$$

注意：移出的位会丢失，除非指令集另有保存规则。

2. 算术移位

通常用于有符号数。

对于补码：

- 算术左移：右侧补0，原有位整体左移。
- 算术右移：左侧补符号位。

例如8位补码：

$$-8 = 11111000$$

算术右移1位：

$$11111100$$

对应：

$$\boxed{-4}$$

算术右移2位：

$$11111110$$

对应：

$$\boxed{-2}$$

注意算术左移可能溢出

例如：

$$01000000 = 64$$

左移1位：

$$10000000$$

按8位补码解释：

$$-128$$

数学上的结果本应为128，但8位补码无法表示。

因此发生溢出。

3. 循环移位

循环移位将移出的位重新移入另一端。

例如：

$$10010110$$

循环左移1位：

$$00101101$$

逻辑右移与算术右移对比

原数据	逻辑右移1位	算术右移1位
01110000	00111000	00111000
11110000	01111000	11111000

记忆：逻辑右移补0，补码算术右移补符号位。

六、定点数加减运算 必会

1. 补码加法

使用补码时：

$$\boxed{[x+y]_{补}=[x]_{补}+[y]_{补}}$$

只保留规定字长的低n位。

例如8位：

$$5+(-3)$$

$$5=00000101$$

$$-3=11111101$$

相加：

```
  00000101
+ 11111101
-----------
1 00000010
```

舍弃第9位进位：

$$00000010=2$$

结果：

$$\boxed{2}$$

2. 补码减法

减法转换为加法：

$$\boxed{x-y=x+(-y)}$$

例如：

$$5-3$$

实际计算：

$$5+(-3)=2$$

注意：

如果已经知道 $[y]_{补}$，求 $[-y]_{补}$ 通常可以对原n位编码**按位取反再加1**。

但对于最小负数，例如8位补码 -128，其相反数128无法用8位有符号数表示。

七、溢出判断 必会

这是408非常典型的选择题和计算题。

1. 什么是溢出?

当数学运算结果超出机器数能够表示的范围，就发生溢出。

8位补码范围：

$$[-128,127]$$

例如：

$$100+50=150$$

150超出表示范围，所以发生溢出。

2. 符号判断法

对于补码加法：

两个同号数相加，结果符号相反，则发生溢出。

例如：

$$100+50$$

都是正数，但8位结果会变成负数。

所以溢出。

相反：

- 正数+负数，不会发生有符号加法溢出。
- 正数+正数，可能正溢出。
- 负数+负数，可能负溢出。

例题

8位补码：

$$01111111+00000001$$

即：

$$127+1$$

二进制结果：

$$10000000$$

解释为：

$$-128$$

因为正数加正数得到了负数，所以：

$$\boxed{OF=1}$$

3. 双符号位法

使用变形补码，即两个符号位。

结论：

- 两个符号位相同：没有溢出。
- 两个符号位不同：发生溢出。

例如：

$$01\cdots$$

表示正溢出。

$$10\cdots$$

表示负溢出。

注意：这里讨论的是双符号位的变形补码运算，不是普通单符号位编码。

4. 进位判断法

设：

- C_{in}：进入符号位的进位。
- C_{out}：符号位向最高位之外产生的进位。

则：

$$\boxed{OF=C_{in}\oplus C_{out}}$$

其中：

$$\oplus$$

表示异或。

也就是说：

两个进位不同，则发生有符号溢出。

5. CF与OF的区别

标志	判断内容
CF	无符号加法进位或无符号减法借位
OF	有符号运算结果是否超出范围

例如8位：

$$255+1$$

无符号结果超出255：

$$CF=1$$

而按补码解释：

$$-1+1=0$$

没有有符号溢出：

$$OF=0$$

注意：减法中CF表示借位的具体约定依指令集而定。

八、无符号数比较与有符号数比较

相同的二进制结果，比较方式可能不同。

1. 无符号比较

通常根据：

- CF
- ZF

等标志判断。

2. 有符号比较

通常需要：

- SF
- OF
- ZF

例如对于某些常见ISA的减法比较结果：

$$x<y$$

有符号条件可以通过：

$$\boxed{SF\neq OF}$$

来判断。

无符号条件通常由借位相关标志判断。

注意：具体条件码定义需要以题目给定的指令集为准。

九、定点乘法 重点

定点乘法主要考查：

- 原码一位乘法
- 补码一位乘法
- 移位与加法
- 乘积位数

1. 乘法的基本原理

二进制乘法可以通过：

部分积相加与移位 实现。

例如：

$$1011_2 \times 0101_2$$

即：

$$11 \times 5$$

展开：

```
    纯文本
      1011
×     0101
-----------
      1011
     0000
    1011
   0000
-----------
    110111
```

$$110111_2 = 55$$

因此：

$$\boxed{11 \times 5 = 55}$$

2. 原码一位乘法

基本思想：

1. 符号位单独处理。
2. 数值部分进行无符号乘法。
3. 根据乘数最低位决定是否加被乘数。
4. 通过移位形成部分积。
5. 最后确定乘积符号。

乘积符号：

$$\boxed{S = S_x \oplus S_y}$$

同号得正，异号得负。

3. 补码一位乘法

常见采用Booth算法思想。

设：

- Q_0：乘数当前最低位。
- Q_{-1}：附加位。

按下面规则处理：

Q_0Q_{-1}	操作
00	不加不减
01	加被乘数
10	减被乘数
11	不加不减

每轮操作后执行算术右移。

记忆口诀

> 01加，10减，00和11不操作。

注意：具体考试题可能使用教材规定的Booth变体，运算寄存器宽度、循环次数及最后一次是否移位，需要严格遵循题目算法。

4. 乘积位数

两个n位无符号整数相乘，完整乘积最多需要：

$$\boxed{2n \text{ 位}}$$

两个n位补码有符号整数相乘，完整乘积也可以用2n位补码表示。

例如：

8位×8位：

$$\boxed{16 \text{ 位乘积}}$$

如果只保留低8位，就可能丢失高位信息。

十、定点除法 重点

定点除法主要考查：

- 恢复余数法
- 不恢复余数法（加减交替法）
- 商和余数的处理
- 除法溢出与除零

1. 恢复余数法

基本思想：

1. 将部分余数减去除数。
2. 判断余数符号。
3. 若余数非负，则商相应位置1。
4. 若余数为负，则恢复余数，商相应位置0。
5. 移位，继续下一轮。

2. 不恢复余数法

核心思想：

根据余数符号决定下一步加除数还是减除数，避免每次都立即恢复余数。

一般规则：

- 余数非负：下一步倾向于减除数。

- 余数为负：下一步倾向于加除数。

不同教材算法的移位、加减顺序略有区别，需要按照题目给出的规则执行。

3. 除法易错点

- 除数为0不能正常完成除法。
- 商可能超出规定机器数范围。
- 整数除法与小数定点除法的运算规则不完全相同。
- 余数符号、修正方式需要根据具体算法判断。

对于你目前以基础分为目标的复习，建议先理解乘除法的原理和运算表，再针对真题出现的具体算法练习竖式，不必一开始把大量不同变体混在一起。

十一、浮点数基本表示 必会

浮点数通常采用科学计数法思想。

一般表示为：

$$\boxed{N = (-1)^S \times M \times R^E}$$

其中：

- S：符号。
- M：尾数。
- R：基数。
- E：阶码。

二进制浮点数通常以2为基数。

1. IEEE 754单精度

32位浮点数：



IEEE 754 binary32（单精度）字段结构

单精度：

- 符号位：1位
- 阶码位：8位
- 尾数字段：23位
- 阶码偏置值：127

对于规格化有限数：

$$\boxed{N = (-1)^S \times (1.F)_2 \times 2^{E-127}}$$

注意：这里的E是阶码字段按无符号整数解释后的值，且 $1 \leq E \leq 254$。

2. IEEE 754双精度

64位浮点数：

字段	位数
符号位	1
阶码	11
尾数字段	52

偏置值：

$$\boxed{1023}$$

规格化有限数：

$$\boxed{N = (-1)^S \times (1.F)_2 \times 2^{E-1023}}$$

单精度与双精度对比

项目	单精度	双精度
总位数	32	64
阶码位数	8	11
尾数字段位数	23	52
有效精度位数（含隐藏位）	24	53
Bias	127	1023

十二、IEEE 754浮点数转换 必会

这是最值得掌握的固定题型之一。

典型例题：将13.25转成IEEE 754单精度

第一步：确定符号位

13.25是正数：

$$S = 0$$

第二步：转换为二进制

整数部分：

$$13 = 1101_2$$

小数部分：

$$0.25 = 0.01_2$$

所以：

$$13.25 = 1101.01_2$$

第三步：规格化

$$1101.01_2$$

移动小数点：

$$\boxed{1.10101_2 \times 2^3}$$

真实阶码：

$$e = 3$$

第四步：计算阶码字段

$$E = e + 127$$

$$E = 3 + 127 = 130$$

130转二进制：

$$\boxed{10000010}$$

第五步：计算尾数字段

规格化尾数：

$$1.10101$$

隐藏最高位1，只保存小数部分：

$$10101$$

补齐23位：

$$\boxed{10101000000000000000000}$$

第六步：拼接

$$S = 0$$

$$E = 10000010$$

$$F = 10101000000000000000000$$

最终：

```
纯文本
0 | 10000010 | 10101000000000000000000
```

二进制整体：

```
纯文本
01000001010101000000000000000000
```

对应十六进制：

$$\boxed{41540000_H}$$

固定步骤

遇到十进制转IEEE 754：

1. 确定符号位。
2. 转换为二进制。
3. 写成 $1.xxx \times 2^e$。
4. 计算阶码 $E = e + Bias$。
5. 去掉隐藏的最高位1。
6. 补齐尾数字段，必要时舍入。
7. 拼接并转换为十六进制。

十三、IEEE 754特殊值 必会

特别注意：不是所有浮点数都使用 $1.F \times 2^{E-Bias}$ 公式。

阶码字段	尾数字段	表示
全0	全0	正零或负零
全0	非0	非规格化数（次正规数）
非全0、非全1	任意	规格化有限数
全1	全0	正无穷或负无穷
全1	非0	NaN

1. 非规格化数

单精度：

$$\boxed{N = (-1)^S \times (0.F)_2 \times 2^{-126}}$$

注意：

- 隐含整数位是0。
- 指数不是 -127，而是 -126。

2. 无穷大

阶码全1，尾数字段全0。

例如：

$$+\infty$$

对应符号位0。

3. NaN

Not a Number，表示非数值结果。

例如浮点运算：

$$0.0/0.0$$

通常产生NaN。

4. 重要数值范围

IEEE 754单精度：

最小正规格化数：

$$\boxed{2^{-126}}$$

最小正非规格化数：

$$\boxed{2^{-149}}$$

最大有限正数：

$$\boxed{(2-2^{-23})\times 2^{127}}$$

十四、浮点数加减法 必会

浮点加减法的固定流程：

1. 对阶
↓
2. 尾数加减
↓
3. 结果规格化
↓
4. 舍入处理
↓
5. 判断上溢、下溢及特殊情况

1. 对阶

两浮点数相加，首先要使阶码一致。

通常：

小阶向大阶看齐。

例如：

$$A = 1.101 \times 2^4$$

$$B = 1.110 \times 2^2$$

要把B的阶码调整为4。

阶码增加2，尾数右移2位：

$$B = 0.01110 \times 2^4$$

于是：

$$A = 1.10100 \times 2^4$$

$$B = 0.01110 \times 2^4$$

2. 尾数相加

$$1.10100 + 0.01110$$

得到：

$$10.00010$$

所以：

$$A+B=10.00010\times 2^4$$

3. 规格化

$$10.00010\times 2^4$$

小数点左移一位，阶码加1：

$$\boxed{1.000010\times 2^5}$$

4. 舍入

如果尾数超过能够保存的位数，需要舍入。

IEEE 754默认舍入模式通常为：

就近舍入，遇到正好居中时取偶数（roundTiesToEven）。

5. 判断溢出

规格化、舍入后检查指数范围。

- 指数过大：可能上溢。
- 指数太小：可能需要使用非规格化数，或进一步下溢。

注意：阶码低于最小规格化指数，不代表一定直接变为0。

必背口诀

> 小阶对大阶，尾数做加减；结果要规格化，最后舍入查溢出。

十五、浮点数乘除法 重点

1. 浮点乘法

两个浮点数：

$$A=(-1)^{S_1}M_1 2^{e_1}$$

$$B=(-1)^{S_2}M_2 2^{e_2}$$

乘积：

$$\boxed{AB=(-1)^{S_1\oplus S_2}(M_1M_2)2^{e_1+e_2}}$$

固定步骤：

1. 符号异或。
2. 尾数相乘。
3. 真实阶码相加。
4. 规格化。
5. 舍入。
6. 检查异常和溢出。

如果直接使用偏置阶码字段：

$$\boxed{E = E_1 + E_2 - Bias}$$

这个公式针对正常有限数的阶码初步计算，后续规格化仍可能修改阶码。

2. 浮点除法

$$\boxed{\frac{A}{B} = (-1)^{S_1 \oplus S_2} \frac{M_1}{M_2} 2^{e_1 - e_2}}$$

若使用偏置阶码：

$$\boxed{E = E_1 - E_2 + Bias}$$

仍需进一步规格化、舍入并处理特殊值。

十六、浮点数精度与误差 重点

1. 为什么浮点数存在误差?

因为二进制浮点数只能精确表示有限精度内的某些数。

例如：

$$0.1_{10}$$

无法表示为有限位二进制小数。

因此浮点数通常只能存储其近似值。

2. 浮点数精度

单精度有效二进制位数：

$$\boxed{24 \text{ 位}}$$

双精度：

$$\boxed{53 \text{ 位}}$$

这里包括规格化数隐藏的最高位1。

对应十进制有效数字大致为：

- 单精度：约7位。
- 双精度：约15～16位。

3. ULP

ULP可理解为给定数值附近相邻可表示浮点数之间的间距。

例如单精度规格化数在：

$$[1, 2)$$

范围内，间距为：

$$\boxed{2^{-23}}$$

在：

$$[2,4)$$

范围内，间距为：

$$\boxed{2^{-22}}$$

浮点数的绝对间距通常随数量级增大而增大。

十七、ALU与加法器 重点

1. ALU

算术逻辑单元，Arithmetic Logic Unit。

主要完成：

- 加法
- 减法
- 与、或、异或等逻辑运算
- 比较
- 移位相关操作（具体可能由独立移位器完成）

2. 一位全加器

输入：

- A
- B
- 低位进位 C_{in}

输出：

- 和 S
- 进位 C_{out}

求和：

$$\boxed{S = A \oplus B \oplus C_{in}}$$

进位：

$$\boxed{C_{out} = AB + (A \oplus B)C_{in}}$$

3. 串行进位加法器

也称行波进位加法器。

特点：

- 结构简单。
- 高位进位需要等待低位进位。
- 位数越多，进位传播延迟可能越大。

4. 并行进位加法器

提前计算进位，减少等待。

定义：

$$G_i = A_i B_i$$

$$P_i = A_i \oplus B_i$$

那么：

$$\boxed{C_{i+1} = G_i + P_i C_i}$$

其中：

- G_i：进位产生信号。
- P_i：进位传递信号。

展开：

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

这样可以通过组合逻辑提前计算多个进位。

5. 加法器性能比较

类型	优点	缺点
行波进位加法器	简单、面积较小	进位传播较慢
先行进位加法器	运算速度较快	电路较复杂
多级先行进位加法器	适合较宽字长	设计更加复杂

十八、408常见题型与固定步骤

这一部分是最值得直接背的内容。

题型1：原码、反码、补码转换

题目特征：

- 给定整数真值。
- 给定机器字长。
- 求原码、反码、补码。

固定步骤：

1. 将绝对值转换为二进制。
2. 补齐数值位。
3. 根据正负设置符号位。
4. 负数反码：数值位取反。
5. 负数补码：反码加1。

特殊注意：

- 最小负数不能使用同位数原码表示。
- 补码负数的符号位参与编码，不是独立数值符号。

题型2：补码转真值

看到补码：

1. 判断最高位。
2. 最高位0：直接转十进制。
3. 最高位1：将整个编码当无符号数转换，再减 2^n。

$$\boxed{x = U - 2^n}$$

其中U为补码位模式的无符号解释值。

题型3：判断溢出

看到补码加减法：

1. 将减法变为加法。
2. 完成n位运算。
3. 判断操作数及结果符号。
4. 或用符号位进位异或：

$$\boxed{OF = C_{in} \oplus C_{out}}$$

题型4：移位运算

1. 先确定有符号还是无符号。
2. 确定逻辑移位还是算术移位。
3. 左移低位补0。
4. 逻辑右移高位补0。
5. 补码算术右移高位补符号位。
6. 最后检查位数截断与溢出。

题型5：IEEE 754十进制转浮点

1. 确定符号。
2. 十进制转二进制。
3. 规格化。
4. 计算偏置阶码。
5. 去隐藏位。
6. 补齐或舍入尾数。
7. 拼接编码。

题型6：IEEE 754浮点转十进制

1. 拆出S、E、F。
2. 检查是否特殊值。
3. 计算真实阶码。
4. 恢复有效数字。
5. 代入公式：

$$\boxed{N = (-1)^S (1.F)_2 2^{E-Bias}}$$

该公式只适用于规格化有限数。

题型7：浮点数加减法

直接记：

$$\boxed{\text{对阶} \rightarrow \text{尾数运算} \rightarrow \text{规格化} \rightarrow \text{舍入} \rightarrow \text{异常检查}}$$

题型8：浮点数乘除法

乘法：

$$e = e_1 + e_2$$

除法：

$$e = e_1 - e_2$$

符号：

$$S = S_1 \oplus S_2$$

最后进行规格化、舍入及异常处理。

十九、考前公式速查表

知识点	公式
n位无符号整数范围	$0 \sim 2^n - 1$
n位原码整数范围	$-(2^{n-1} - 1) \sim 2^{n-1} - 1$
n位补码整数范围	$-2^{n-1} \sim 2^{n-1} - 1$
负数补码	$2^n - \|x\|$
常规移码	$x + 2^{n-1}$
补码负数求真值	$U - 2^n$
补码加法溢出	$C_{in} \oplus C_{out}$
单精度Bias	127
双精度Bias	1023
单精度有效位数	24
双精度有效位数	53
浮点乘法阶码	$e_1 + e_2$
浮点除法阶码	$e_1 - e_2$
浮点乘除法符号	$S_1 \oplus S_2$
全加器求和	$A \oplus B \oplus C_{in}$
全加器进位	$AB + (A \oplus B)C_{in}$

二十、易错点总结

常见错误	正确理解
8位补码10000000表示负零	表示-128
补码范围与原码相同	补码多表示一个最小负数
CF=OF	两者含义不同
负数逻辑右移要补1	逻辑右移补0
补码算术右移补0	补符号位
相同二进制编码代表相同数值	还需看有符号或无符号
IEEE 754阶码直接等于真实指数	要减偏置值
单精度尾数字段23位，所以精度23位	规格化数包含隐藏位，共24位
IEEE 754阶码全0仍按隐藏位1处理	非规格化数隐藏整数位是0
阶码全1表示无穷大	尾数字段非0时表示NaN
浮点加法先加尾数再对阶	必须先对阶
浮点数的小数都可以精确表示	部分十进制小数只能近似存储

二十一、基础自测：8道题

数据的表示和运算：基础练习

0/8 已答

1. 8位补码11111011对应的十进制真值是多少?

A. -5

B. -4

C. 251

D. -123

2. 8位补码表示范围是什么?

A. -127～127

B. -128～127

C. -128～128

D. 0～255

3. 8位补码11110000算术右移1位后是什么?

A. 01111000

B. 11111000

C. 11110001

D. 01110000

4. 两个8位补码正数01111111与00000001相加，OF是多少?

A. 0

B. 1

C. 无法判断

D. 取决于CF

5. IEEE 754单精度的阶码偏置值是什么?

A. 128

B. 127

C. 1023

D. 255

6. 单精度规格化数的有效二进制精度是多少位?

A. 23

B. 24

C. 32

D. 31

7. 浮点数加减法的正确基本顺序是?

A. 加减→对阶→舍入

B. 对阶→尾数运算→规格化→舍入

C. 规格化→对阶→加减

D. 舍入→对阶→加减

8. 补码一位乘法Booth算法中，Q0Q-1=10时通常如何处理?

A. 加被乘数

B. 减被乘数

C. 不加不减

D. 直接结束

二十二、针对你目前复习情况的拿分顺序

这一章的知识点比较多，但不必全部投入同等精力。

优先顺序	复习内容	要达到的水平
1	原码、反码、补码、移码	看到编码立即转换
2	补码加减与溢出	会直接计算CF、OF
3	移位、符号扩展、类型转换	不在基础选择题丢分
4	IEEE 754编码	会十进制与十六进制相互转换
5	浮点数加减运算	记住对阶、规格化、舍入步骤
6	全加器、先行进位	掌握基本公式
7	定点乘除算法	理解原理并练典型真题

这一章最值得反复练习的是四类题：

- **补码转换题**：按固定字长转换正负整数。
- **溢出判断题**：看符号变化或计算进位异或。
- **IEEE 754编码题**：按符号、阶码、尾数依次处理。
- **浮点加减题**：先对阶，再运算、规格化、舍入。

对于你目前的目标，建议先把这四类基础题练到能够独立解答，再逐步增加Booth乘法、除法和进位电路分析的练习量。这样比一开始平均分配时间更容易形成稳定的得分能力。